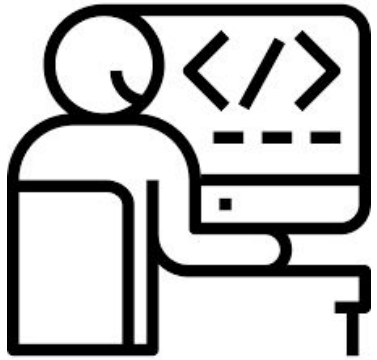


Algoritmos y Programación I

AyPI – **Temas de la clase pasada**

- Estructura de control FOR
- Estructuras de control WHILE

AyPI – **Temas de la clase de hoy**

- Modularización
- Procedimientos
- Comunicación entre módulos

AyPI – **MODULARIZACIÓN**

Los problemas del mundo real implican:

- **Complejidad**
- **Extensión**
- **Modificaciones**

Los tratamos de resolver con:

- **Abstracción**
- **Descomposición**
- **Independencia Funcional**

AyPI – **MODULARIZACIÓN**

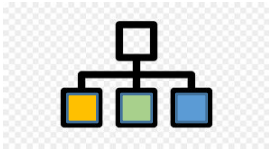
MODULARIZAR

Significa dividir un problema en partes funcionalmente independientes, que encapsulen operaciones y datos.

No se trata simplemente de subdividir el código de un sistema de software en bloques con un número de instrucciones dado.

Separar en funciones lógicas con datos propios y datos de comunicación perfectamente especificados.

AyPI – **MODULARIZACIÓN**



MODULARIZAR

Cada subproblema está al mismo nivel de detalle.

Cada subproblema puede resolverse independientemente.

Las soluciones de los subproblemas puede combinarse para resolver el problema original.

AyPI – **MODULARIZACIÓN**

MÓDULO

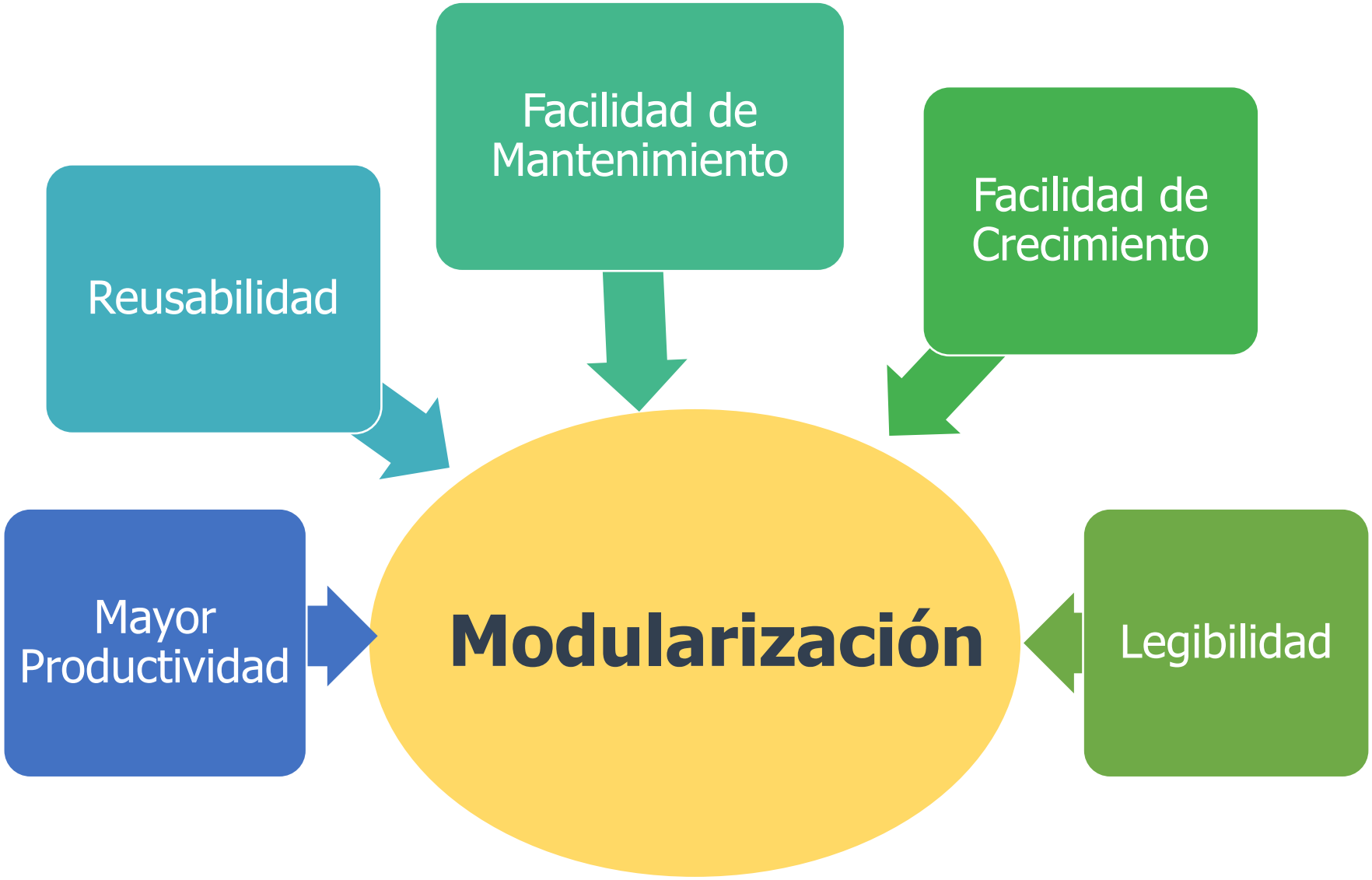
Tarea específica bien definida se comunican entre sí adecuadamente y cooperan para conseguir un objetivo común.

Encapsula acciones o tareas.

En cada uno de ellos se puede representar alguno de los objetivos principales del problema a resolver.

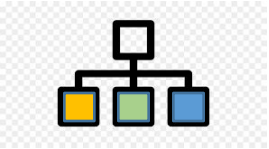
AyPI – **MODULARIZACIÓN**

VENTAJAS

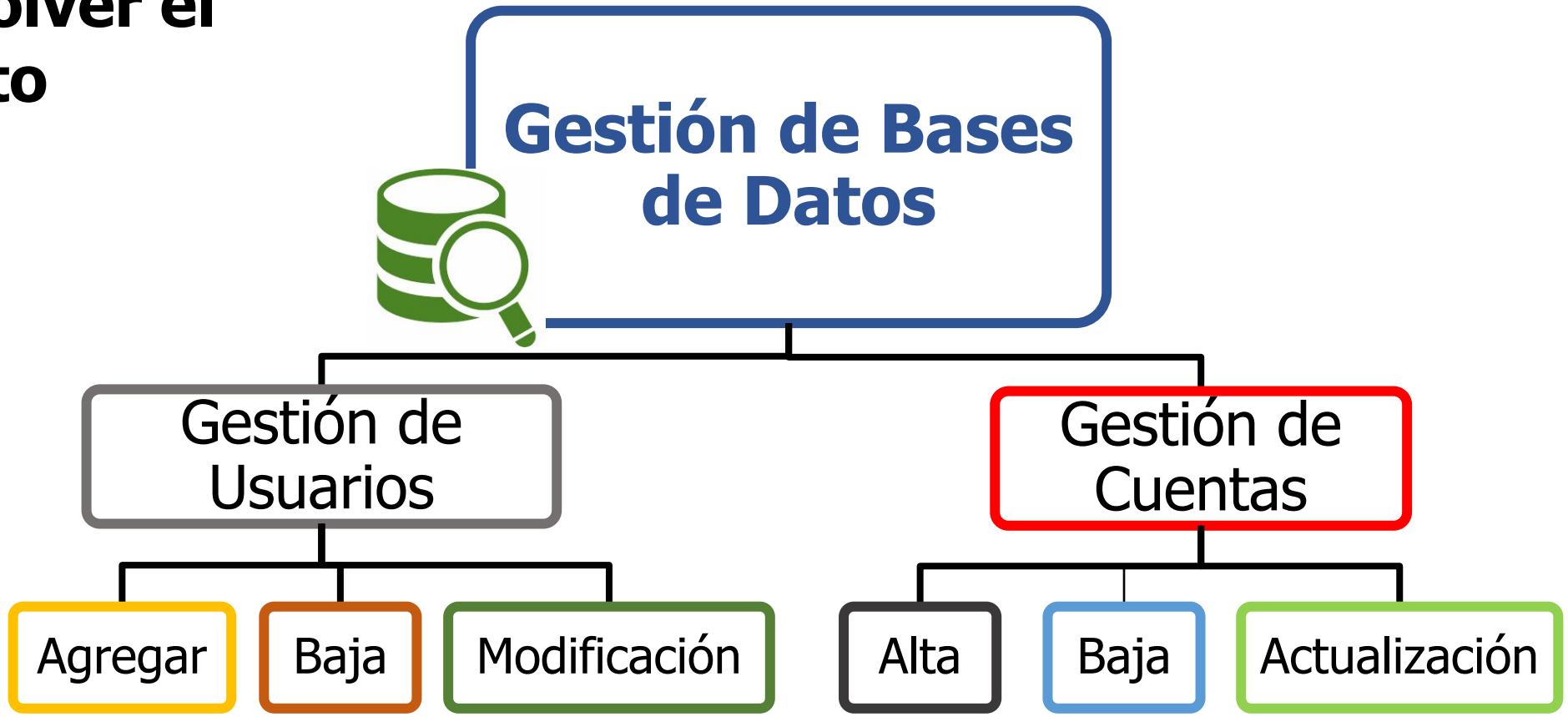


AyPI – **MODULARIZACIÓN**

VENTAJAS

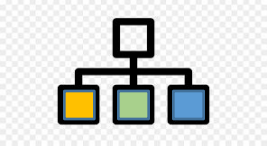


Supongamos que tenemos que resolver el siguiente proyecto



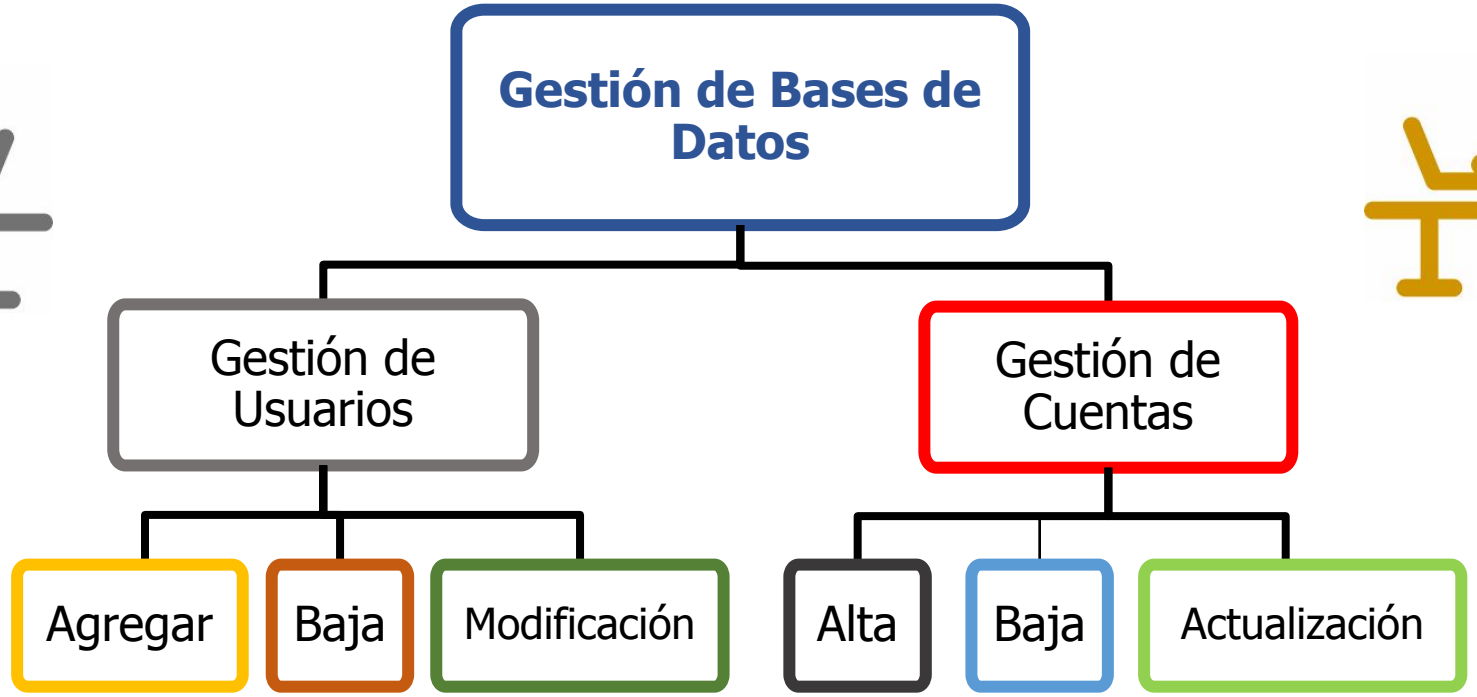
AyPI – **MODULARIZACIÓN**

VENTAJAS



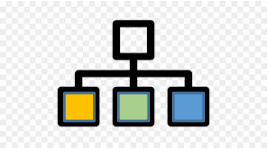
Al dividir un sistema de software en módulos funcionalmente independientes, un equipo de desarrollo puede trabajar simultáneamente en varios módulos, incrementando la productividad (es decir reduciendo el tiempo de desarrollo global del sistema).

Mayor Productividad



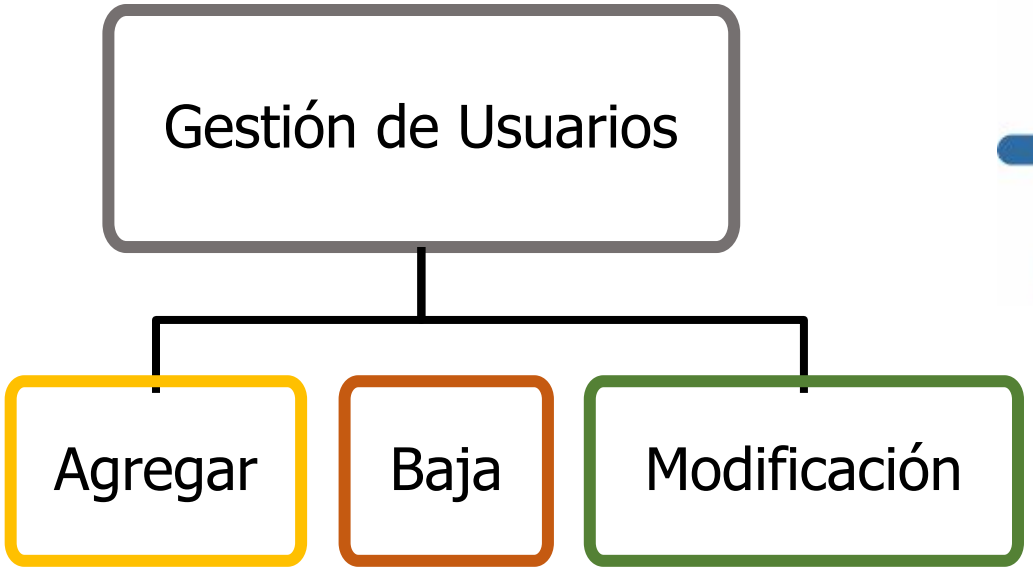
AyPI – **MODULARIZACIÓN**

VENTAJAS



Un objetivo fundamental de la modularización es la reusabilidad, es decir la posibilidad de utilizar repetidamente parte del desarrollo en el mismo o en otros problemas.

Reusabilidad

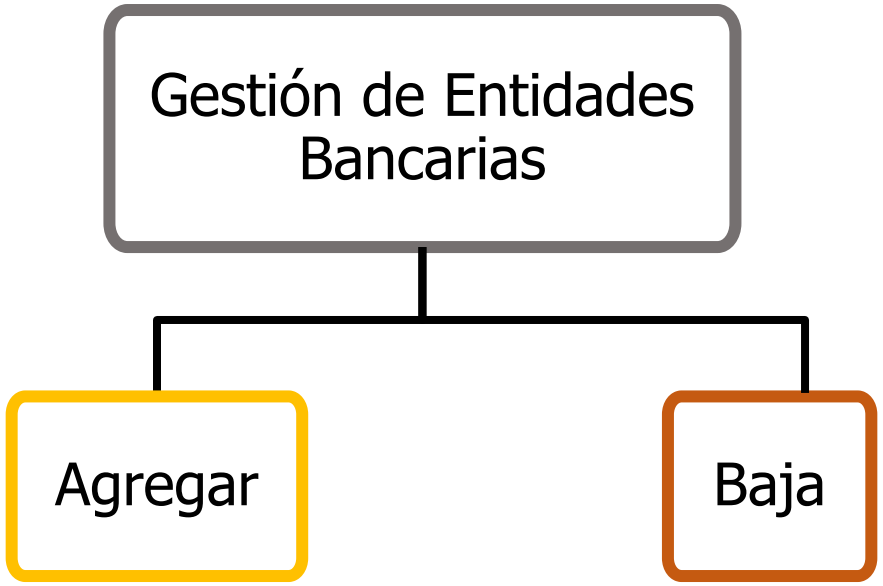


AyPI – **MODULARIZACIÓN**

VENTAJAS

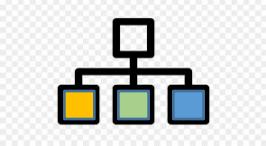
Los sistemas de software reales crecen (es decir aparecen con el tiempo nuevos requerimientos del usuario). La modularización permite disminuir los riesgos y costos de incorporar nuevas prestaciones a un sistema en funcionamiento.

Facilidad de crecimiento



AyPI – **MODULARIZACIÓN**

VENTAJAS

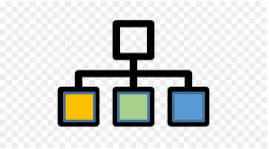


Un efecto de la modularización es una mayor claridad para leer y comprender el código fuente. El ser humano maneja y comprende con mayor facilidad un número limitado de instrucciones directamente relacionadas.

Legibilidad

AyPI – **MODULARIZACIÓN**

PROCEDIMIENTOS



```
Program nombre;
type
  ...
  procedure nombre(...);
   var ...
   begin
   end;
var ...
begin
end.
```

¿Cómo son?
¿Cómo se declaran?
¿Cómo se usan?

PROCEDIMIENTO

Conjunto de instrucciones que realizan una tarea específica y retorna 0, 1 ó más valores.

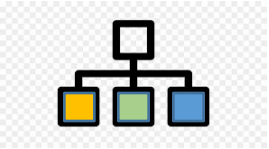
```
procedure nombre(...);
var
  ....
begin
  ....
end;
```

Variables locales

Código del procedimiento

¿Cómo se invocan?

AyPI – **MODULARIZACIÓN** **PROCEDIMIENTOS**

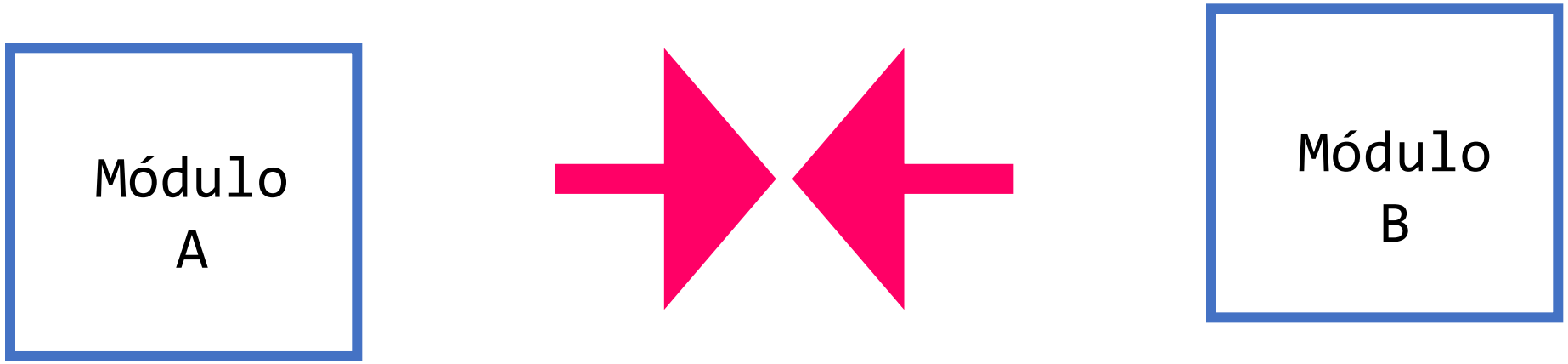


INVOCACIÓN

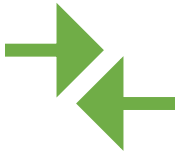
```
Program uno;
Const
  ....
Type
  ....
procedure auxiliar(...);
Var ...

begin
 ...
end;

Var
 ....
Begin
  auxiliar(...);
End.
```

Por su nombre

COMUNICACIÓN ENTRE MÓDULOS



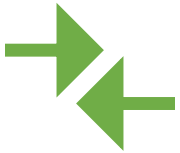
Parámetros



PARÁMETROS

Los datos compartidos entre módulos se deben especificar como parámetros que se transmiten de un módulo a otro.

El uso de parámetros implica que los datos exclusivos de un módulo NO son ni "visibles" ni utilizables por otros módulos (ocultamiento).



PARÁMETROS – ¿Cómo vamos a trabajar?

1
- Se analiza para cada módulo entonces: ¿cuáles son los datos propios? y ¿cuáles son los datos compartidos?

2
- Los datos propios se declararan locales al módulo.

3
- Los datos compartidos se declararán como parámetros.

Parámetros por valor

Parámetros por referencia

PARÁMETRO POR VALOR

Un dato de entrada por valor se utiliza para enviarle valores a un módulo. Éste recibe sobre una variable local la copia del valor proveniente de otro módulo (o del programa principal).

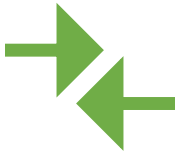
Con él puede realizar operaciones y/o cálculos. Si lo modifica no producirá ningún cambio ni tampoco tendrá incidencia fuera del módulo.

¿Cómo se declaran?

¿Cómo se usan?

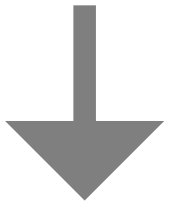
PARÁMETRO POR VALOR

```
procedure uno (nombre1: tipo; nombre2: tipo);

  var
     …

  Begin

    Uso de los parámetros con nombre1 y nombre2

  End;
```



PARÁMETRO POR VALOR

```
Program porValor;
procedure uno (num: integer);
Begin
    if (num = 7)  then
      num:= num + 1;
    write (num);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write (x);
end.
```

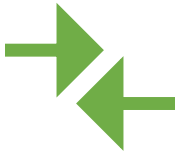


Dentro del procedimiento uno, el parámetro num copia el valor enviado por x (variable del programa)

¿Cómo funciona?

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



```
Program porValor;

procedure uno (num: integer);
Begin
    if (num = 7)  then
      num:= num + 1;
    write (num);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write(x);
end.
```

¿Qué pasa si después de llamar al procedimiento uno en el programa imprimo num?

Procedimiento uno
Variables locales
Parámetros

num = ~~8~~

Imprime 8

Programa ppal
Variables de prog

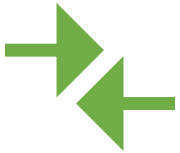
x = 7

Imprime 7

MEMORIA

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



```
Program porValor;

procedure uno (num: integer);
Begin
    if (num = 7)  then
      num:= num + 1;
    write (num);
end;
var
   num: integer;
begin
  num:= 7;
  uno (num);
  write(num);
end.
```

¿Qué pasa si después de llamar al procedimiento uno en el programa imprimo num?

Procedimiento uno
Variables locales
Parámetros

num = ~~8~~

Imprime 8

Programa ppal
Variables de prog

num = 7

Imprime 7

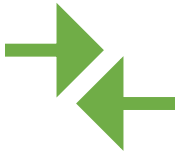
MEMORIA

AyPI – **MODULARIZACIÓN**

```
Program porValor;

procedure uno (x: integer);
Begin
    if (x = 7)  then
      x:= x + 1;
    write (x);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write(x);
end.
```

¿Qué valores imprimen?

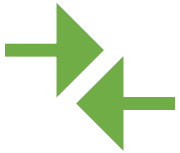
COMUNICACIÓN



```
Program porValor;
procedure uno (num: integer);
Var
 x:integer;
Begin
    if (num = 7) then
       num:= num + 1;
    x:= num;
    write (num); write (x);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write(x);
end.
```

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



```
Program equivalentes1;

procedure uno (x, y: integer);
Begin
    if (x = 7)  then
      y:= y + 1;
    write (y);
end;
var
   x, y: integer;
begin
  x:= 7;  y:= 8;
  uno (y, x);
  write(x);
end.
```

¿Qué valores imprimen?

```
Program equivalentes2;

procedure uno (x, y: integer);
Begin
    if (x = 7)  then
      y:= y + 1;
    write (y);
end;
var
   x: integer;
begin
  x:= 7;
  uno (8, x);
  write(x);
end.
```

PARÁMETRO POR REFERENCIA

La comunicación por referencia implica que el módulo comparte la memoria de otra variable conocida en otros módulos del sistema.

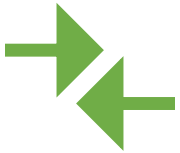
Puede operar con ella y su valor original dentro del módulo, y las modificaciones que se produzcan se reflejan en los demás módulos que comparten la variable.

¿Cómo se declaran?

¿Cómo se usan?

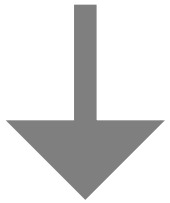
PARÁMETRO POR REFERENCIA

```
procedure uno (var nombre1: tipo; var nombre2: tipo);

  var
     …

  Begin

    Uso de los parámetros con nombre1 y nombre2

  End;
```



PARÁMETRO POR REFERENCIA

```
Program porReferencia;
procedure uno (var num: integer);
Begin
    if (num = 7)  then
      num:= num + 1;
    write (num);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write(x);
end.
```

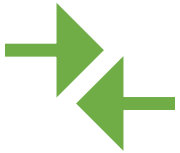


Dentro del procedimiento uno, el parámetro num comparte la dirección de memoria con x (variable del programa)

¿Cómo funciona?

AyPI – **MODULARIZACIÓN**

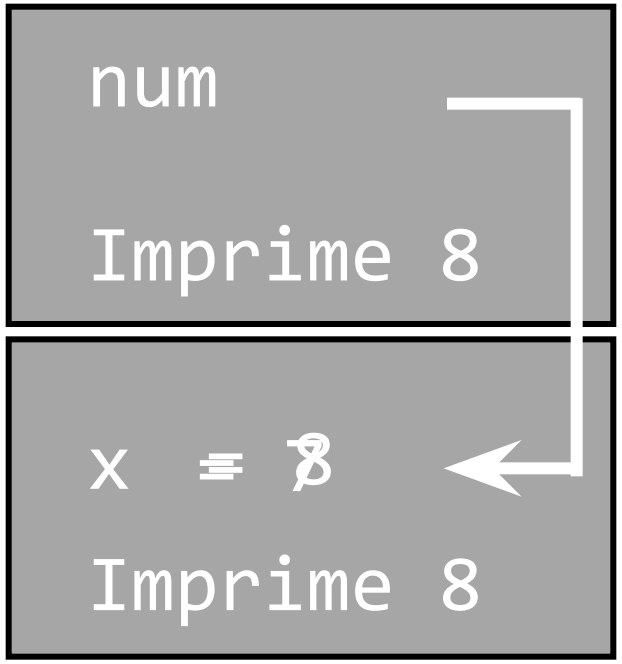
COMUNICACIÓN



```
Program porReferencia;

procedure uno (var num: integer);
Begin
    if (num = 7)  then
       num:= num + 1;
    write (num);
end;
var
   x: integer;
begin
  x:= 7;
  uno (x);
  write(x);
end.
```

Procedimiento uno
Variables locales
Parámetros

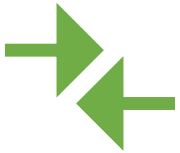
Programa ppal
Variables de prog



MEMORIA

AyPI – **MODULARIZACIÓN**

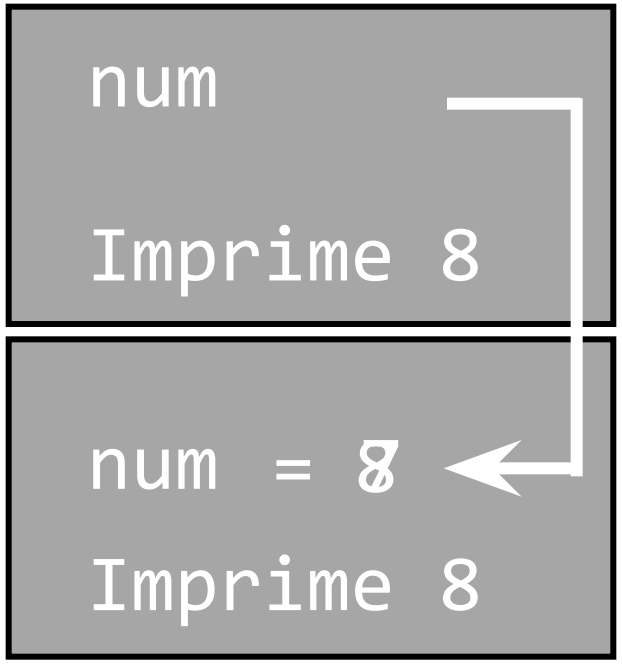
COMUNICACIÓN



```
Program porReferencia;

procedure uno (var num: integer);
Begin
    if (num = 7)  then
       num:= num + 1;
    write (num);
end;
var
   num: integer;
begin
  num:= 7;
  uno (num);
  write(num);
end.
```

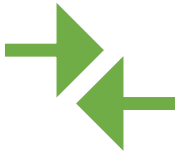
Procedimiento uno
Variables locales
Parámetros

Programa ppal
Variables de prog



MEMORIA

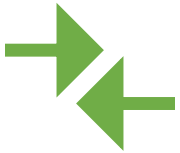
AyPI – **MODULARIZACIÓN** COMUNICACIÓN



```
Program IMC;

procedure calcularIMC(var imc: real; peso, altura: real);
Begin
    imc:= peso / (altura * altura);
end;

var
   peso, altura, imc: real;     i: integer;
begin
for i:= 1 to 10 do begin
   read (peso); read (altura);
   calcularIMC(imc, peso, altura);
   write(imc);
   end;
end.
```

AyPI – **MODULARIZACIÓN**

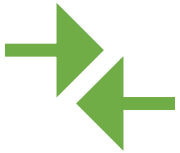
COMUNICACIÓN



- El número y tipo de los argumentos utilizados en la invocación a un módulo deben coincidir con el número y tipo de parámetros del encabezamiento del módulo.
- Un parámetro por valor debiera ser tratado como una variable de la cual el módulo hace una copia y la utiliza localmente. Algunos lenguajes permiten la modificación local de un parámetro por valor, pero toda modificación realizada queda en el módulo en el cual el parámetro es utilizado.
- El número y tipo de los argumentos utilizados en la invocación a un módulo deben coincidir con el número y tipo de parámetros del encabezamiento del módulo.

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN

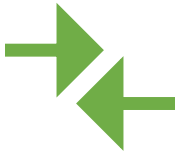


¿Compila?

```
Program uno;

Var
    x:integer;
    c:char;

procedure ejemplo (var a:integer; j:char);
begin
  // código del procedimiento ejemplo
end;

begin
     c:='a';
     ejemplo (15,c);
end.
```

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



¿Compila?

```
Program uno;

Var
    x:integer;

procedure ejemplo (var a:integer; j:char);
begin
  // código del procedimiento ejemplo
end;

begin
   x:=25;
   ejemplo (x,'p');
end.
```

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN

Escriba un programa que lea una serie de números enteros (hasta ingresar el valor -1) e imprima para cada número si múltiplo de 10. Modularice la solución.

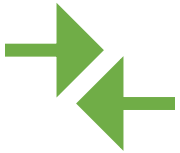
- ¿Qué es lo que se modulariza?
- ¿Qué parámetros necesita el módulo?
- ¿Cómo determino si es múltiplo de 10?

```
17
3
20
8
100
-1
```

```
False
False
True
False
True
```

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



```
program multiplos;
procedure esMultiplo (num:integer; var resultado:boolean);
var  …
begin
  …
end;
Var
 valor:integer; multiplo: boolean;
Begin
  read (valor);
  while valor <> -1 do begin
    esMultiplo(valor, multiplo);
    write (‘¿El valor ‘, valor, ’ es multiplo de 10?’, multiplo);
    read(valor);
  end;
End.
```

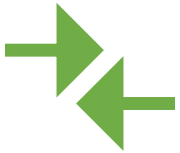
Copia el contenido de valor

Comparte la memoria con multiplo

¿Cómo se implementa esMultiplo?

AyPI – **MODULARIZACIÓN**

COMUNICACIÓN



```
procedure esMultiplo (num:integer; var resultado:boolean);
var
   resto: integer;
Begin
  resto:= num mod 10;

  if resto = 0 then
      resultado:= true
   else
      resultado:= false;
end;
```

Copia el valor recibido

Devuelve si es múltiplo de 10 o no.

AyPI – **Actividad**

Implemente un módulo que reciba las dimensiones de un terreno (frente y fondo) y retorne el área y perímetro correspondiente.

Implemente un módulo que reciba el área de un terreno, y el valor del metro cuadrado en pesos y retorne la valuación fiscal en pesos.

Implemente un módulo que reciba la valuación fiscal en pesos de un terreno y una tasa de cambio a dólares y retorne la valuación fiscal en dólares.

Implemente un programa que procese la información de los 15 terrenos de una inmobiliaria. El programa debe leer el valor del metro cuadrado en pesos y la tasa de cambio a dólares. Luego, para cada terreno se debe leer los valores del frente y fondo. El programa debe imprimir el área, el perímetro y la valuación fiscal en dólares.